Formal Languages and Automata Theory

Unit 1: Finite Automata

An alphabet Σ is a finite, non-empty set of symbols. A string is a finite sequence of symbols from Σ, and a language is any set of strings over Σ. The empty string is written ε.

1.1 Deterministic Finite Automata (DFA)

A DFA is a 5-tuple M = (Q, Σ, δ, q0, F) where Q is a finite set of states, Σ the input alphabet, δ: Q × Σ → Q the transition function, q0 ∈ Q the start state and F ⊆ Q the set of accepting states. M accepts w if the extended transition function δ*(q0, w) lands in F.
Example: the DFA over {0,1} that accepts strings with an even number of 1s needs two states, q_even (start, accepting) and q_odd. Reading 1 swaps states; reading 0 stays put.

1.2 Nondeterministic Finite Automata (NFA)

An NFA allows δ: Q × (Σ ∪ {ε}) → P(Q). A string is accepted if at least one computation path ends in an accepting state. Every NFA has an equivalent DFA, built by the subset construction; in the worst case the DFA has 2^n states.

1.3 Minimisation

Two states are equivalent if no string distinguishes them. The table-filling (Myhill–Nerode) algorithm marks distinguishable pairs and merges the rest, giving the unique minimal DFA.